SIMATS ENGINEERING

CO2 ASSESSMENT TOOL 3

ROHAN P T

192424112

Course Code: CSA1401

Course Name: Compiler Design

Case Study 1 — Parsing Nested Components in a Web Framework (React-style)

Scenario: A web framework like React parses nested components.

Input:

```
<id><id></id></id>
```

Grammar:

```
S → < id > S </ id > S | ε
```

(i) Explanation of Nested Parsing

JSX/HTML-style markup is fundamentally a recursively nested structure: every opening tag must be balanced by a matching closing tag of the same name, and any content between them is itself a (possibly empty) sequence of further nested elements. The grammar S → <id> S </id> S | ε captures exactly this: the first S after the opening tag represents the children nested inside the current element, and the second S (after the closing tag) represents sibling elements that follow at the same level. Because the rule is self-embedding (S appears inside its own expansion), a single production can describe components nested to any depth — exactly like a React component tree, where a <Card> can contain another <Card>, which can contain a <Button>, and so on. Parsing such structures is naturally done with a recursive-descent (top-down) parser, since each call to the S-procedure mirrors one level of the component tree, and the run-time call stack automatically tracks how deeply the parser is nested — conceptually the same job performed by React's reconciler when it walks a nested element tree.

(ii) Demonstration of Parsing

Treating <, id, >, and the closing-tag marker </ as distinct lexical tokens (a typical tag-aware lexer would emit '</' as one token rather than '<' followed by '/'), FIRST(S) = { <, ε } and the ε-alternative is chosen only when the lookahead is a closing tag or end of input — never '<' — so the grammar is LL(1) and can be parsed with simple one-token lookahead, no backtracking.

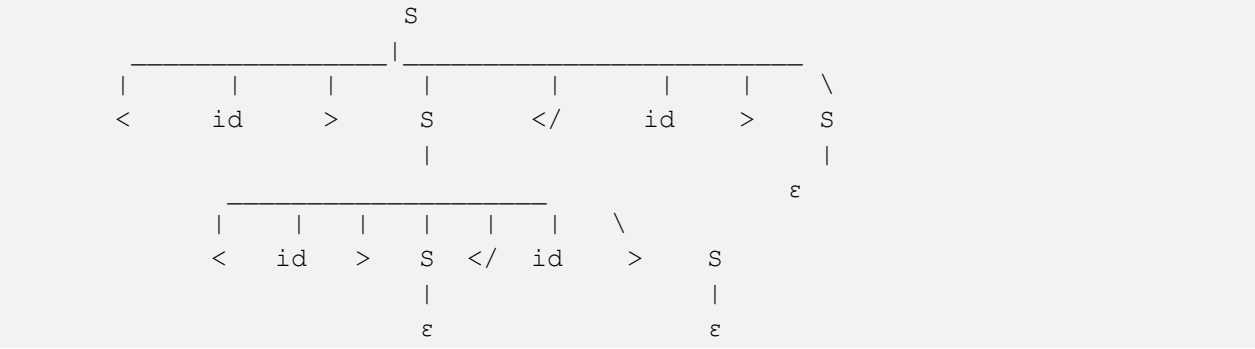
```
procedure S():
    if lookahead == '<':
        match('<'); match(id); match('>')      // opening tag, e.g. <id>
```

```
        S()                                         // parse nested children
        match('</'); match(id); match('>')          // matching closing tag
        S()                                         // parse following siblings
    else:
        return                                      // ε-production: no more
elements here
```

Step	Call	Lookahead	Action
1	S() [outer]	<	match '<' 'id' '>' — consumes outer opening tag “<id>”
2	→ S() [inner]	<	match '<' 'id' '>' — consumes inner opening tag “<id>”
3	→→ S() [children of inner]	</	ε-production, return (inner tag has no children)
4	← back in inner S()	</	match '</' 'id' '>' — consumes inner closing tag “</id>”
5	→ S() [siblings after inner]	</	ε-production, return (no sibling after inner tag)
6	← back in outer S()	</	match '</' 'id' '>' — consumes outer closing tag “</id>”
7	→ S() [siblings after outer]	$	ε-production, return (input exhausted)

Every token of the input “<id><id></id></id>” is consumed exactly once and the top-level call returns cleanly, so the string is ACCEPTED.

(iii) Parse Tree



Reading the tree top to bottom mirrors the DOM/component hierarchy React would build: the outer <id> node has exactly one child, the inner <id> node, which itself has no children and no following siblings — matching a component tree of depth 2.

(iv) Identifying Mismatched Tags

The bare grammar treats every tag name as the generic terminal 'id', so by itself it cannot tell one tag name from another. In practice, the parser is augmented with a semantic side-stack of actual tag-name values: on every opening tag the matched name is pushed; on every closing tag the name is popped and compared against the value in the closing tag. Two kinds of mismatch are then detected:

- Name mismatch — e.g. input : after opening and , the parser meets first; popping the stack gives 'span', which does not equal 'div', so the parser reports “Expected closing tag but found ”.
- Unbalanced tags — if a closing tag is seen while the name-stack is empty (an extra </id> with no matching opener), or if input ends while the name-stack is still non-empty (an opening tag never closed), the parser reports “unmatched closing tag” or “unclosed element” respectively.

For the given input “<id><id></id></id>” the name-stack pushes 'id' (outer) then 'id' (inner); the first </id> pops 'id' and matches; the second </id> pops the remaining 'id' and matches; the stack ends empty exactly when input ends — so no mismatch is reported and the nesting is confirmed well-formed.

(v) Suggested Improvements

- Extend the grammar so tag names are captured as attributed terminals (id.name) rather than a single generic 'id', and thread a name-stack through the parser's semantic actions so mismatched-tag detection is part of the formal parsing process, not an external patch.
- Add explicit panic-mode error recovery: on a mismatch, skip tokens up to the next '<' or '</' so the parser can report multiple independent errors in one pass instead of aborting at the first one.
- Support self-closing tags (e.g. <id/>) with an additional production S → < id /> S, common in JSX for leaf components.
- Support tag attributes (e.g. <id key="...">) by extending the opening-tag production with an optional attribute-list non-terminal.

Case Study 2 — Parsing Continuous Sensor Streams in an IoT Gateway

Scenario: An IoT gateway processes continuous sensor data.

Input Stream:

```
< < data > data >
```

Grammar:

```
S → < S > S | data
```

Challenges: continuous streaming; partial data arrival.

(i) Parsing Strategy

Unlike the previous grammar, S here has no ε-alternative — every S must eventually bottom out at a 'data' token, and S → <S>S requires two full sub-parses. FIRST(S) = { <, data } and the two alternatives start with disjoint tokens, so the grammar is LL(1): a single lookahead token always decides which rule to apply, with no backtracking. This is ideal for an IoT gateway, where sensor frames arrive continuously over a socket or MQTT topic and the parser cannot afford to rescan or buffer unboundedly — a one-token-lookahead, table-driven (stack-based, non-recursive) predictive parser is used instead of a call-stack recursive-descent parser, because an explicit stack can be serialized, paused, and resumed as new bytes trickle in, whereas a paused native call stack cannot.

(ii) Demonstration of Parsing

Predictive table: M[S,<] = S → <S>S, M[S,data] = S → data, M[S,>] and M[S,$] are undefined (errors), since neither alternative can be selected on '>' or on empty input.

Step	Stack (bottom→top)	Input Remaining	Action
1	$ S	< < data > data >	M[S,<] = S→<S>S; push S,>,S,< (top='<')
2	$ S > S <	< data > data >	match '<' (outer bracket); consume token
3	$ S > S	< data > data >	top=S, la='<' → M[S,<]=S→<S>S; push S,>,S,<
4	$ S > S > S <	data > data >	match '<' (inner bracket); consume token

Step	Stack (bottom→top)	Input Remaining	Action
5	$ S > S > S	data > data >	top=S, la='data' → M[S,data]=S→data; push 'data'
6	$ S > S > data	data > data >	match 'data'; consume token
7	$ S > S >	> data >	match '>' (closes inner bracket); consume token
8	$ S > S	data >	top=S, la='data' → M[S,data]=S→data; push 'data'
9	$ S > data	data >	match 'data'; consume token
10	$ S >	>	match '>' (closes outer bracket); consume token
11	$ S	(buffer empty)	top=S, no token available — PARSER SUSPENDS, awaiting more data

Ten of the eleven steps consume all six tokens of the given stream (< < data > data >) exactly as intended; the parser is then left needing one more S to satisfy the trailing S of the outermost S→<S>S rule, but the buffer is currently empty.

(iii) Handling Incomplete Input

This is precisely the 'partial data arrival' challenge: the grammar is not nullable, so the outstanding S on the stack genuinely requires more tokens — either another 'data' reading or a further nested <...> frame — before the current statement can be completed. A streaming-aware parser must not treat an empty buffer the same as a syntax error. Instead it: (a) freezes the current explicit stack ($, S) and the fact that zero tokens have been matched against it yet; (b) returns control to the network layer with a 'need more input' status rather than 'accept' or 'reject'; (c) resumes exactly where it left off — by reloading the saved stack — as soon as the next chunk of bytes/tokens arrives from the sensor link, with no re-scanning of already-consumed tokens.

(iv) Detecting Errors

A robust gateway parser distinguishes three outcomes rather than a single accept/reject:

- Waiting (not an error): the stack is non-empty and expects a symbol, but the buffer is simply empty because the next network chunk has not yet arrived — parsing is suspended, as in step 11 above.

- Genuine syntax error: a token arrives that has no table entry for the current top-of-stack symbol — for example, top=S with lookahead '>' (M[S,>] is undefined), or top='<' expecting a literal '<' but the incoming token is 'data'. Either is reported immediately as "unexpected token, expected < or data" (or the specific literal expected), since no future input can make the current derivation valid.
- Truncated stream: the sender explicitly signals end-of-transmission (e.g. socket closed) while the stack is still non-empty, as would happen if no further bytes ever arrive after step 11. This differs from ordinary waiting because it is a confirmed, permanent absence of data; it should be reported as "incomplete / truncated sensor packet" rather than silently discarded, so the fault can be logged or the reading re-requested.

(v) Suggested Improvements

- Introduce an explicit length-prefixed or delimiter-terminated framing protocol (e.g. a byte count or a unique end-of-record marker) so the gateway knows in advance exactly how much data to expect, removing the ambiguity between 'waiting' and 'truncated'.
- Persist the parser's explicit stack and buffer pointer as a small serializable state object per connection, so thousands of concurrent sensor streams can each be parsed incrementally without dedicating a blocked thread/call-stack to each one.
- Add a per-stream timeout that discards or flags a suspended parse if no continuation arrives within an expected window, to reclaim resources from dead or disconnected sensors.
- Attach sequence numbers or checksums to each frame so reordered, duplicated, or corrupted fragments are detected independently of the grammar-level parse.
- For richer future sensor formats, migrate from a hand-rolled predictive parser to a table-driven LALR/streaming-JSON-style parser, which generalizes more easily to additional data types beyond a single 'data' terminal.

Case Study 3 — Parsing Multiple Embedded Languages in an IDE

Scenario: An IDE supports multiple embedded languages.

Input:

```
id = id + id
```

Grammar:

```
S → id = E
E → E + T | T
T → id
```

Challenges: mixed syntax rules; multiple parsing layers.

(i) Integration of Parsing Across Layers

Modern IDEs regularly host more than one grammar in the same source file — for example a host scripting language whose right-hand sides are ordinary arithmetic expressions, or a template language embedding SQL or HTML fragments. This is naturally modelled with a layered grammar: an outer statement layer (S → id = E) recognizes the assignment shape and delegates everything after '=' to an inner, independently maintained expression layer (E → E + T | T, T → id). Because E is left-recursive it is a natural fit for a bottom-up (LR/precedence) sub-parser, while the outer assignment shape is trivial enough for simple top-down pattern matching; many real IDEs use exactly this hybrid architecture — a lightweight top-down driver that recognizes statement boundaries and hands well-defined token spans to a dedicated expression-grammar parser (sometimes even a different parser generator or language server per embedded language), with the two layers communicating only through clearly defined start/end token boundaries and a shared token stream/position map so that editor features like syntax highlighting and error-squiggles line up correctly across the boundary.

(ii) Demonstration of Parsing

To parse top-down, the indirect left recursion in E is first eliminated:

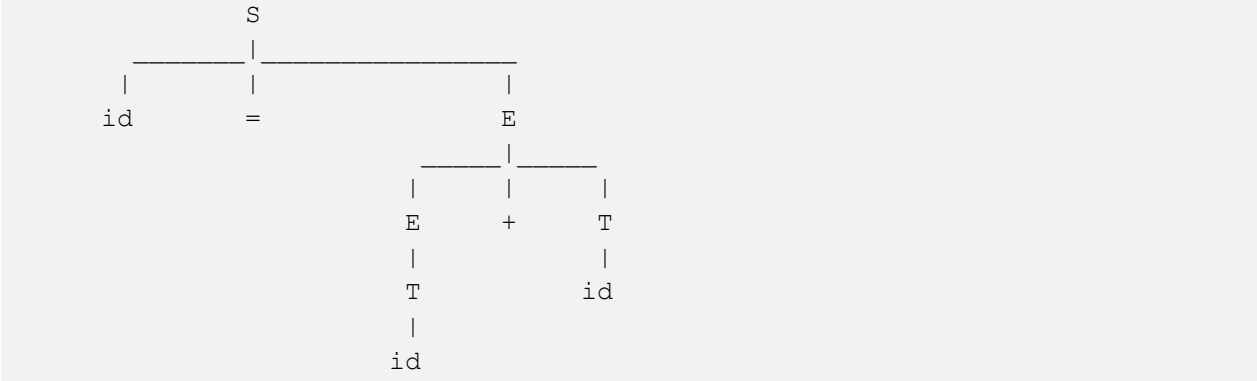
```
E  → T E'
E' → + T E' | ε
T  → id
```

Step	Call	Lookahead	Action
1	S()	id	match('id') — the assigned variable
2	S()	=	match('=')
3	→ E() → T()	id	match('id') — first operand
4	← E() → E'()	+	lookahead '+' → match('+')
5	→ T()	id	match('id') — second operand
6	→ E'()	$	lookahead ∉ {+} → ε-production, return

All tokens (id, =, id, +, id) are consumed and every call returns normally, so "id = id + id" is ACCEPTED.

(iii) Parse Tree

Using the original (unmodified, still unambiguous) grammar directly, since the input has only one possible grouping:



The tree shows the outer statement layer contributing the top two children (id and '='), while the entire right-hand structure belongs to the embedded expression layer — exactly the modular split described in part (i).

(iv) Identifying Syntax Errors

Typical layer-specific errors this grammar can pinpoint:

- Missing assignment operator, e.g. 'id id + id': after matching the first 'id', S expects '=' but finds 'id' — the statement layer reports "expected '=' but found 'id'".
- Expression starting with an operator, e.g. 'id = + id': E (via T) expects an 'id' to begin the expression but finds '+' — the expression layer reports "expected 'id' but found '+'".

- Dangling operator, e.g. 'id = id +': after matching '+', T again expects 'id' but the input is exhausted — reported as “unexpected end of input, expected 'id' after '+'”.
- Trailing extra tokens, e.g. 'id = id + id id': once E is fully parsed (ending after the third 'id'), the statement layer expects end-of-statement, but an extra 'id' remains with no operator joining it — reported as “unexpected token 'id', expected end of statement”, since 'id' is not in FOLLOW(E').

Because each error is raised inside the specific procedure (S, E, E', or T) that detected it, the IDE can attribute the error to the correct language layer and underline exactly the offending token span.

(v) Suggested Improvements

- Report errors together with which layer raised them (statement vs. expression) so the IDE can route the diagnostic to the correct language service and offer layer-appropriate quick-fixes.
- Add synchronizing tokens (e.g. ';' or newline) for panic-mode recovery at the statement layer, and operator/identifier tokens for the expression layer, so one bad token does not cascade into unrelated downstream errors.
- Support incremental re-parsing: when a user edits only the expression after '=', re-parse just that embedded region instead of the whole file, which is essential for real-time IDE responsiveness.
- Maintain a unified source-position map across layers so that error ranges, hover tooltips, and go-to-definition all resolve correctly even though two independent grammars/parsers are involved.
- Extend E/T with a factor level (F → (E) | id) and additional operators (*, /) to support a fuller expression sub-language, and add a semantic-analysis pass (type checking, scope/symbol-table resolution) on top of the syntax layer, since IDE-grade language support needs more than CFG-level recognition alone.